

Chapitre 2

Vocabulaire ensembliste

2.1 Éléments de logique

2.1.1 Motivation : L'ordinateur face à l'ambiguïté

Dans la vie de tous les jours, le langage humain est riche mais souvent imprécis. L'intonation ou le contexte suffisent à nous faire comprendre. À l'inverse, en mathématiques et en programmation, une instruction ne peut souffrir d'aucune ambiguïté. Si vous programmez le système de validation d'un site de e-commerce, l'ordinateur doit savoir exactement dans quelles conditions appliquer une réduction. La logique formelle est le langage universel qui permet de traduire des règles métier complexes en assertions mathématiques irréfutables (vraies ou fausses, sans entre-deux).

2.1.2 Approche par problème : Le contrat trompeur

Problème : Une assurance auto propose la clause suivante : *"Vous êtes remboursé si le conducteur a plus de 25 ans OU s'il a son permis depuis plus de 3 ans ET qu'il n'a pas eu d'accident."* Un conducteur de 26 ans, ayant son permis depuis 1 an et ayant eu un accident réclame son remboursement. L'assurance doit-elle payer ?

Résolution : La phrase en français est ambiguë à cause de l'absence de parenthèses.

- Interprétation 1 : (Plus de 25 ans OU Permis > 3 ans) ET (Pas d'accident). Ici, le conducteur a eu un accident, l'assertion totale est Fausse, il n'est pas remboursé.
- Interprétation 2 : (Plus de 25 ans) OU (Permis > 3 ans ET Pas d'accident). Ici, le conducteur a plus de 25 ans, la première condition est Vraie, l'assertion totale est Vraie, il est remboursé !

Pour éviter les litiges, les mathématiques imposent l'usage strict de connecteurs logiques hiérarchisés.

2.1.3 Définitions des connecteurs logiques

Une **proposition** (ou assertion) est un énoncé mathématique qui prend une et une seule valeur de vérité : Vrai (V) ou Faux (F). Soient P et Q deux propositions.

- **Négation (NON)** : Notée $\neg P$. Elle est vraie si P est fausse.
- **Conjonction (ET)** : Notée $P \wedge Q$. Elle est vraie uniquement si P et Q sont toutes les deux vraies.
- **Disjonction (OU)** : Notée $P \vee Q$. Elle est vraie si *au moins* l'une des deux est vraie. (Il s'agit d'un "ou" inclusif).

- **Implication** ($\implies$) : $P \implies Q$ ("Si P alors Q "). Elle n'est fausse que dans un seul cas : quand P est vraie et Q est fausse. Elle est logiquement équivalente à $(\neg P) \vee Q$.
- **Équivalence** ($\iff$) : $P \iff Q$ signifie que P et Q ont toujours la même valeur de vérité.

2.1.4 Les Quantificateurs

Pour formuler des propriétés sur les éléments d'un ensemble, on utilise des quantificateurs :

- **Le quantificateur universel** ($\forall$) : "Pour tout" ou "Quel que soit". Exemple : $\forall x \in \mathbb{R}, x^2 \geq 0$.
- **Le quantificateur existentiel** ($\exists$) : "Il existe au moins un". Exemple : $\exists x \in \mathbb{R}, x^2 = 2$.

Règle de négation : Prendre la négation d'une phrase quantifiée inverse les quantificateurs.

$$\neg(\forall x \in E, P(x)) \iff \exists x \in E, \neg P(x)$$

$$\neg(\exists x \in E, P(x)) \iff \forall x \in E, \neg P(x)$$

2.1.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Tables de vérité - Facile) : Démontrer que l'implication $P \implies Q$ est équivalente à sa contraposée $\neg Q \implies \neg P$. *Correction* : On dresse la table de vérité. Dans les cas où (P, Q) valent respectivement $(V, V), (V, F), (F, V), (F, F)$, l'expression $P \implies Q$ vaut V, F, V, V . L'expression $\neg Q \implies \neg P$ (qui se lit "Si $\neg Q$ est vrai, alors $\neg P$ doit être vrai") donne exactement les mêmes résultats : V, F, V, V . Les deux assertions sont donc équivalentes.

Exemple 2 (Négation d'une phrase courante - Intermédiaire) : Écrire la négation de : "Toutes les voitures de cette marque sont rouges ou noires." *Correction* : Formellement : $\forall v \in \text{Voitures}, (\text{Rouge}(v) \vee \text{Noire}(v))$. La négation transforme le $\forall$ en $\exists$, et par la loi de De Morgan, la négation du OU devient un ET : $\exists v \in \text{Voitures}, (\neg \text{Rouge}(v) \wedge \neg \text{Noire}(v))$. En français : "Il existe au moins une voiture de cette marque qui n'est ni rouge ni noire."

Exemple 3 (Raisonnement par l'absurde - Difficile) : Démontrer que $\sqrt{2}$ est irrationnel ($\sqrt{2} \notin \mathbb{Q}$). *Correction* : On suppose par l'absurde la négation de l'assertion, c'est-à-dire que $\sqrt{2} \in \mathbb{Q}$. Il existe donc deux entiers p et q (avec $q \neq 0$) premiers entre eux tels que $\sqrt{2} = \frac{p}{q}$. En élevant au carré : $2 = \frac{p^2}{q^2} \implies p^2 = 2q^2$. p^2 est pair, donc p est pair. On peut écrire $p = 2k$. En remplaçant : $(2k)^2 = 2q^2 \implies 4k^2 = 2q^2 \implies q^2 = 2k^2$. q^2 est donc pair, ce qui implique que q est pair. Nous avons trouvé que p et q sont tous deux pairs (divisibles par 2), ce qui contredit notre hypothèse initiale qu'ils sont premiers entre eux. L'hypothèse de départ est donc fausse : $\sqrt{2}$ est irrationnel.

2.1.6 Exercices pratiques avec Python

Les opérateurs logiques (**and**, **or**, **not**) sont les piliers des structures conditionnelles en algorithmique. Voici comment vérifier des règles métier strictes à l'aide de la logique booléenne.

```
1 def est_eligible_bourse(etudiant):
2     """
```

```

3     Regle : Un etudiant est eligible s'il a plus de 14 de moyenne
4     OU (s'il a plus de 12 ET qu'il est boursier sur criteres
5         sociaux).
6     """
7     moyenne = etudiant['moyenne']
8     critere_social = etudiant['critere_social']
9
10    # Application stricte de la logique formelle avec parentheses
11    if moyenne >= 14 or (moyenne >= 12 and critere_social == True):
12        return True
13    else:
14        return False
15
16    # Test des propositions
17    etudiant_A = {'nom': 'Alice', 'moyenne': 13, 'critere_social':
18        False}
19    etudiant_B = {'nom': 'Bob', 'moyenne': 12.5, 'critere_social': True
20        }
21
22    print(f"Alice est eligible : {est_eligible_bourse(etudiant_A)}") #
23        Renvoie False
24    print(f"Bob est eligible : {est_eligible_bourse(etudiant_B)}") #
25        Renvoie True

```

2.1.7 Exercices à faire

1. **Exercice 1** : À l'aide d'une table de vérité, démontrer la loi de De Morgan : $\neg(P \wedge Q) \iff (\neg P) \vee (\neg Q)$.
2. **Exercice 2** : Traduire la définition suivante de la limite d'une suite avec des quantificateurs : "Pour tout réel ε strictement positif, il existe un rang N à partir duquel la distance entre u_n et L est strictement inférieure à ε ". Écrire ensuite sa négation.
3. **Exercice 3** : Démontrer par contraposée l'assertion suivante pour tout entier n : "Si $n^2 - 1$ n'est pas divisible par 8, alors n est pair".

2.2 Éléments de la théorie des ensembles

2.2.1 Motivation : L'agrégation de données extraites

Lors de l'extraction automatisée d'offres d'emploi par web scraping sur des portails professionnels (comme *Tunisie Travail* par exemple), les scripts récupèrent souvent de vastes listes d'annonces classées par catégories. Pour croiser ces informations — par exemple, identifier les annonces qui requièrent à la fois des compétences en analyse de données et en vision par ordinateur — il est indispensable de modéliser ces listes comme des ensembles mathématiques. L'algèbre ensembliste permet de formaliser ces intersections, unions et exclusions pour concevoir des algorithmes de filtrage robustes et garantis sans doublons.

2.2.2 Approche par problème : Le paradoxe de l'appartenance

Problème : Considérons l'ensemble de toutes les offres d'emploi d'une plateforme, noté E . Une offre spécifique pour un poste de développeur, notons-la o_1 , appartient à E . On écrit $o_1 \in E$. Si l'on prend maintenant la catégorie regroupant toutes les offres en "Data Analysis", notée D . Peut-on écrire $D \in E$?

Résolution : C'est une erreur fondamentale en algèbre. La catégorie D n'est pas une offre d'emploi unique ; c'est une *sous-collection* d'offres. Il faut donc distinguer strictement l'**appartenance** (un élément dans un ensemble, symbole $\in$) de l'**inclusion** (un ensemble contenu dans un autre, symbole $\subset$). On écrira rigoureusement $D \subset E$.

2.2.3 Définitions fondamentales

Définition 2.1 (Inclusion et Égalité) : Soient A et B deux ensembles.

— A est inclus dans B , noté $A \subset B$, si tout élément de A est aussi un élément de B .

$$\forall x, (x \in A \implies x \in B)$$

— Deux ensembles sont égaux ($A = B$) s'ils ont exactement les mêmes éléments. Pour le prouver en exercice, on démontre presque toujours la **double inclusion** : ($A \subset B$) et ($B \subset A$).

Définition 2.2 (Ensemble des parties) : L'ensemble des parties d'un ensemble E , noté $\mathcal{P}(E)$, est l'ensemble dont les éléments sont tous les sous-ensembles possibles de E . Si $E = \{a, b\}$, alors $\mathcal{P}(E) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$. *Remarque :* L'ensemble vide, noté $\emptyset$, est inclus dans n'importe quel ensemble.

2.2.4 Opérations sur les ensembles

Soient A et B deux parties d'un ensemble de référence E .

- **Union** ($A \cup B$) : Éléments appartenant à A ou à B . ($x \in A \cup B \iff x \in A \vee x \in B$).
- **Intersection** ($A \cap B$) : Éléments appartenant à A et à B . ($x \in A \cap B \iff x \in A \wedge x \in B$).
- **Complémentaire** ($C_E(A)$ ou $\bar{A}$) : Éléments de E n'appartenant pas à A . ($x \in \bar{A} \iff x \notin A$).
- **Différence** ($A \setminus B$) : Éléments de A qui ne sont pas dans B . ($A \setminus B = A \cap \bar{B}$).

2.2.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Opérations simples - Facile) : Soit $E = \{1, 2, 3, 4, 5\}$, $A = \{1, 2, 3\}$ et $B = \{3, 4\}$. Déterminer $A \cap B$, $A \cup B$, et $A \setminus B$. *Correction :* $A \cap B = \{3\}$ (l'élément commun). $A \cup B = \{1, 2, 3, 4\}$ (on fusionne sans répéter le 3). $A \setminus B = \{1, 2\}$ (les éléments de A privés de ceux de B).

Exemple 2 (Preuve d'inclusion - Intermédiaire) : Montrer que pour tous ensembles A et B , on a $A \cap B \subset A$. *Correction :* Soit $x \in A \cap B$. Par définition de l'intersection, cela signifie que la proposition ($x \in A \wedge x \in B$) est vraie. Par conséquent, $x \in A$ est vrai. Tout élément de $A \cap B$ appartient bien à A . Donc $A \cap B \subset A$.

Exemple 3 (Lois de De Morgan pour les ensembles - Difficile) : Démontrer que $\overline{A \cup B} = \bar{A} \cap \bar{B}$. *Correction :* Procédons par équivalence logique en utilisant les résultats

de la Section I. Soit $x \in E$.

$$\begin{aligned}x \in \overline{A \cup B} &\iff \neg(x \in A \cup B) \\&\iff \neg(x \in A \vee x \in B)\end{aligned}$$

D'après les lois de De Morgan en logique :

$$\begin{aligned}&\iff \neg(x \in A) \wedge \neg(x \in B) \\&\iff x \notin A \wedge x \notin B \\&\iff x \in \overline{A} \wedge x \in \overline{B} \\&\iff x \in \overline{A} \cap \overline{B}\end{aligned}$$

L'égalité ensembliste est ainsi démontrée par équivalence.

2.2.6 Exercices pratiques avec Python

Python dispose du type `set`, spécialement conçu pour l'algèbre ensembliste. Il est particulièrement efficace pour croiser des résultats sans avoir à programmer de complexes boucles imbriquées.

```
1  # Identifiants d'offres d'emploi recuperees sur differentes
   # categories d'un site
2  offres_data_analysis = {"ID101", "ID102", "ID103", "ID105"}
3  offres_computer_vision = {"ID103", "ID104", "ID105", "ID106"}
4
5  # 1. Intersection : Offres exigeant simultanement les deux
   # competences
6  offres_cibles = offres_data_analysis.intersection(
   offres_computer_vision)
7  # Syntaxe alternative plus concise : offres_data_analysis &
   offres_computer_vision
8  print(f"Offres correspondantes aux deux : {offres_cibles}")
9
10 # 2. Union : Fusion des deux extractions en supprimant les doublons
11 toutes_offres = offres_data_analysis.union(offres_computer_vision)
12 print(f"Total des annonces uniques extraites : {len(toutes_offres)}")
13
14 # 3. Difference : Offres en Data Analysis sans rapport avec la
   # Vision
15 offres_pures_data = offres_data_analysis - offres_computer_vision
16 print(f"Offres exclusivement Data Analysis : {offres_pures_data}")
```

2.2.7 Exercices à faire

1. **Exercice 1** : Soient A , B et C trois ensembles. Démontrer la distributivité de l'intersection sur l'union : $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.
2. **Exercice 2** : La différence symétrique de deux ensembles, notée $A \Delta B$, est l'ensemble des éléments qui appartiennent à A ou à B , mais pas aux deux à la fois. Montrer que $A \Delta B = (A \cup B) \setminus (A \cap B)$.

3. **Exercice 3** : Sachant que $\mathcal{P}(E)$ est l'ensemble des parties de E , déterminer $\mathcal{P}(\emptyset)$ et $\mathcal{P}(\{1\})$.

2.3 Ensembles finis et dénombrement

2.3.1 Motivation : L'explosion combinatoire en informatique

Le dénombrement est l'art de compter mathématiquement, sans avoir à énumérer chaque cas un par un. En sécurité informatique (cryptographie, mots de passe), en optimisation d'algorithmes, ou en analyse de données, il est indispensable de savoir calculer la taille d'un espace de recherche. Savoir qu'un algorithme de "brute force" devra tester des milliards de combinaisons justifie la recherche de solutions plus élégantes et explique la notion d'explosion combinatoire.

2.3.2 Approche par problème : L'ordre a-t-il de l'importance ?

Problème : Vous disposez d'un groupe de 10 étudiants développeurs.

1. Cas A : Vous devez élire un président, un trésorier et un secrétaire.
2. Cas B : Vous devez former une équipe de 3 développeurs pour un projet.

Combien y a-t-il de choix possibles dans chaque cas ?

Résolution : Dans le Cas A, l'ordre de tirage importe : (Ali=Président, Sara=Trésorière) est différent de (Sara=Présidente, Ali=Trésorier). C'est ce qu'on appelle un **arrangement**. Dans le Cas B, l'équipe Ali, Sara, Karim est rigoureusement identique à l'équipe Sara, Karim, Ali. L'ordre n'a pas d'importance. C'est une **combinaison**. Le dénombrement nous fournit les formules pour calculer ces deux cas distincts instantanément.

2.3.3 Définitions fondamentales

Définition 3.1 (Cardinal) : On dit qu'un ensemble E est fini s'il contient un nombre fini d'éléments. Ce nombre d'éléments est appelé le cardinal de E , noté $\text{Card}(E)$ ou $|E|$. Exemple : Si $E = \{a, b, c\}$, alors $|E| = 3$.

Théorème (Principe d'inclusion-exclusion) : Soient A et B deux ensembles finis. Le cardinal de leur union est donné par :

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Remarque : Si A et B sont disjoints ($A \cap B = \emptyset$), alors $|A \cup B| = |A| + |B|$.

Définition 3.2 (Produit cartésien) : Le produit cartésien de deux ensembles A et B , noté $A \times B$, est l'ensemble des couples (x, y) tels que $x \in A$ et $y \in B$.

$$|A \times B| = |A| \times |B|$$

2.3.4 Les outils de dénombrement

Soit un ensemble fini E de cardinal n , et un entier k tel que $1 \leq k \leq n$.

- **Permutations ($n!$)** : C'est le nombre de façons d'ordonner tous les éléments de E .

$$P_n = n! = 1 \times 2 \times \cdots \times n$$

- **Arrangements** (A_n^k) : C'est le nombre de façons de choisir et d'ordonner k éléments parmi n . (L'ordre compte).

$$A_n^k = \frac{n!}{(n-k)!}$$

- **Combinaisons** ($\binom{n}{k}$) : C'est le nombre de sous-ensembles à k éléments que l'on peut former à partir de E . (L'ordre ne compte pas).

$$\binom{n}{k} = \frac{A_n^k}{k!} = \frac{n!}{k!(n-k)!}$$

2.3.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Cardinal d'une union - Facile) : Dans une promotion de 100 étudiants, 60 étudient Python, 50 étudient Java, et 20 étudient les deux. Combien d'étudiants étudient au moins l'un des deux langages ? *Correction* : On cherche $|P \cup J|$.

$$|P \cup J| = |P| + |J| - |P \cap J| = 60 + 50 - 20 = 90$$

Il y a 90 étudiants. (Il en reste donc 10 qui n'étudient ni l'un ni l'autre).

Exemple 2 (Arrangements ou combinaisons ? - Intermédiaire) : Un code PIN est composé de 4 chiffres distincts (parmi 10). Combien de codes PIN existe-t-il ? *Correction* : L'ordre des chiffres change le code (1234 est différent de 4321). On choisit 4 éléments parmi 10, avec ordre et sans répétition. C'est un arrangement :

$$A_{10}^4 = \frac{10!}{(10-4)!} = \frac{10!}{6!} = 10 \times 9 \times 8 \times 7 = 5040$$

Exemple 3 (Dénombrement complexe - Difficile) : Combien y a-t-il d'anagrammes du mot "DATA" ? *Correction* : Le mot contient 4 lettres, on pourrait penser qu'il y a $4! = 24$ permutations. Cependant, la lettre "A" est répétée 2 fois. Permuter le premier "A" avec le deuxième "A" ne change pas le mot. Il faut donc diviser le nombre total de permutations par le nombre de permutations des lettres identiques :

$$\frac{4!}{2!} = \frac{24}{2} = 12$$

2.3.6 Exercices pratiques avec Python

Le module `itertools` de la bibliothèque standard de Python est l'outil parfait pour générer et manipuler les arrangements et les combinaisons.

```

1 import itertools
2 import math
3
4 etudiants = ["Ali", "Sara", "Karim"]
5
6 # 1. Permutations : Tous les ordres possibles de passage à l'oral
7 ordre_passage = list(itertools.permutations(etudiants))
8 print(f"Permutations_({math.factorial(len(etudiants))}_cas):")
9 for cas in ordre_passage:
```

```

10     print(cas)
11
12 # 2. Combinaisons : Former un binome de projet (l'ordre ne compte
    pas)
13 binomes = list(itertools.combinations(etudiants, 2))
14 print(f"\nCombinaisons_(binomes_possibles)_:")
15 for b in binomes:
16     print(b)
17
18 # Verification mathematique du nombre de binomes (3 parmi 3 -> 3! /
    (2!*1!))
19 print(f"Nombre_mathematique_de_binomes_:_{math.comb(3,2)}")

```

2.3.7 Exercices à faire

1. **Exercice 1** : Dans un jeu de 32 cartes, combien y a-t-il de "mains" de 5 cartes possibles ?
2. **Exercice 2** : Démontrer algébriquement la formule de symétrie : $\binom{n}{k} = \binom{n}{n-k}$. Que signifie cette formule concrètement en termes de choix ?
3. **Exercice 3** : Vous devez constituer un mot de passe de 8 caractères comprenant exactement 3 chiffres et 5 lettres. Sachant que vous avez le choix entre 10 chiffres et 26 lettres (sans répétition), combien de mots de passe pouvez-vous créer ? (*Indice : Choisissez d'abord les positions des chiffres*).

2.4 Applications et relations : Ordre, équivalence et quotient

2.4.1 Motivation : Regroupement et classification de données

Lorsqu'un algorithme de vision par ordinateur analyse la vidéo d'un match de football, il détecte des dizaines d'objets (les joueurs). Pour que l'analyse ait un sens, il faut que la machine comprenne que certains joueurs appartiennent à la même équipe. En mathématiques, on ne va pas créer un ensemble pour chaque équipe de manière arbitraire : on va définir une **relation** (ex : "porte un maillot de la même couleur"). Cette relation va naturellement découper l'ensemble de tous les joueurs en sous-groupes disjoints (les équipes). C'est ce que l'on appelle une classe d'équivalence et un ensemble quotient, concepts fondamentaux pour tout algorithme de *clustering* (regroupement) en Data Science.

2.4.2 Approche par problème : Le tri des doublons

Problème : Vous avez écrit un script web scraping pour aspirer des offres d'emploi sur un portail web. Souvent, la même offre est republiée plusieurs fois à des dates différentes. Comment nettoyer cette base de données ?

Résolution : On définit une relation $\mathcal{R}$ entre deux offres A et B : " $A\mathcal{R}B$ si elles ont exactement le même titre et la même entreprise". Cette relation possède trois propriétés évidentes :

- Une offre est identique à elle-même (Réflexivité).

- Si A est identique à B , alors B est identique à A (Symétrie).
- Si A est identique à B , et B identique à C , alors A est identique à C (Transitivité).

Toute relation vérifiant ces trois critères est une **relation d'équivalence**. Elle permet de regrouper toutes les annonces identiques dans un même "sac" (la classe d'équivalence), et de ne garder qu'un seul représentant par sac.

2.4.3 Définitions des Relations

Soit E un ensemble. Une relation binaire $\mathcal{R}$ sur E relie certains éléments de E entre eux. On écrit $x\mathcal{R}y$ pour dire que " x est en relation avec y ".

Définition 4.1 (Relation d'équivalence) : Une relation $\mathcal{R}$ sur E est une relation d'équivalence si elle est :

1. **Réflexive** : $\forall x \in E, x\mathcal{R}x$
2. **Symétrique** : $\forall (x, y) \in E^2, (x\mathcal{R}y \implies y\mathcal{R}x)$
3. **Transitive** : $\forall (x, y, z) \in E^3, ((x\mathcal{R}y \text{ et } y\mathcal{R}z) \implies x\mathcal{R}z)$

Définition 4.2 (Classe d'équivalence et Ensemble Quotient) : Soit $\mathcal{R}$ une relation d'équivalence sur E . La classe d'équivalence d'un élément $x \in E$, notée $\bar{x}$ ou $cl(x)$, est l'ensemble de tous les éléments de E qui sont en relation avec x .

$$\bar{x} = \{y \in E \mid y\mathcal{R}x\}$$

L'ensemble de toutes ces classes forme une **partition** de E (elles recouvrent tout E et sont deux à deux disjointes). Cet ensemble des classes est appelé **Ensemble Quotient**, noté $E/\mathcal{R}$.

Définition 4.3 (Relation d'ordre) : Une relation $\leq$ est une relation d'ordre si elle est réflexive, transitive et **antisymétrique** :

$$\forall (x, y) \in E^2, ((x \leq y \text{ et } y \leq x) \implies x = y)$$

Un ordre est dit *total* si tous les éléments sont comparables ($x \leq y$ ou $y \leq x$). Sinon, il est *partiel*.

2.4.4 Définitions des Applications (Fonctions)

Soit f une application d'un ensemble de départ E vers un ensemble d'arrivée F .

- **Injective** : Deux éléments distincts de E ont toujours des images distinctes dans F .

$$\forall (x, x') \in E^2, (f(x) = f(x') \implies x = x')$$

- **Surjective** : Tout élément de F possède au moins un antécédent dans E .

$$\forall y \in F, \exists x \in E, f(x) = y$$

- **Bijective** : L'application est à la fois injective et surjective. Tout élément de l'arrivée possède un *unique* antécédent. Cela implique l'existence d'une fonction réciproque f^{-1} .

2.4.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Relation d'ordre - Facile) : Dans l'ensemble des parties $\mathcal{P}(E)$, la relation d'inclusion ($\subset$) est-elle une relation d'ordre total ou partiel? *Correction :* C'est un ordre. Mais il est **partiel**. Par exemple, si $E = \{1, 2\}$, les ensembles $A = \{1\}$ et $B = \{2\}$ ne sont pas comparables : on n'a ni $A \subset B$, ni $B \subset A$.

Exemple 2 (Relation d'équivalence et modulo - Intermédiaire) : Sur $\mathbb{Z}$, on définit $x \mathcal{R} y \iff x - y$ est un multiple de 3. (On note $x \equiv y \pmod{3}$). Quelles sont les classes d'équivalence? *Correction :* Il y a exactement 3 classes d'équivalence, correspondant aux restes possibles de la division euclidienne par 3 : $\bar{0} = \{\dots, -3, 0, 3, 6, \dots\}$ $\bar{1} = \{\dots, -2, 1, 4, 7, \dots\}$ $\bar{2} = \{\dots, -1, 2, 5, 8, \dots\}$ L'ensemble quotient $\mathbb{Z}/3\mathbb{Z}$ est donc $\{\bar{0}, \bar{1}, \bar{2}\}$.

Exemple 3 (Preuve de bijectivité - Difficile) : Soit $f : \mathbb{R} \setminus \{1\} \rightarrow \mathbb{R} \setminus \{2\}$ définie par $f(x) = \frac{2x+1}{x-1}$. Montrer que f est bijective. *Correction :* On va montrer que pour tout $y \in \mathbb{R} \setminus \{2\}$, l'équation $f(x) = y$ admet une unique solution $x \in \mathbb{R} \setminus \{1\}$.

$$\frac{2x+1}{x-1} = y \iff 2x+1 = y(x-1) \iff 2x - yx = -y - 1 \iff x(2-y) = -y - 1$$

Puisque $y \neq 2$, on peut diviser par $(2-y)$:

$$x = \frac{-y-1}{2-y} = \frac{y+1}{y-2}$$

Cette expression est unique et bien définie (le dénominateur ne s'annule pas). f est donc bijective, et sa réciproque est $f^{-1}(y) = \frac{y+1}{y-2}$.

2.4.6 Exercices pratiques avec Python

Les classes d'équivalence sont le fondement de l'opération GROUP BY en bases de données. En Python, on utilise souvent des dictionnaires pour construire l'ensemble quotient (partitionner la donnée).

```
1 # Simulation de donnees (ex: resultats d'un algorithme d'analyse
   # video de football)
2 # Chaque joueur est detecte avec une coordonnee et la couleur
   # dominante de son maillot
3 joueurs = [
4     {"id": 1, "couleur": "Rouge"},
5     {"id": 2, "couleur": "Bleu"},
6     {"id": 3, "couleur": "Rouge"},
7     {"id": 4, "couleur": "Bleu"}
8 ]
9
10 # Creation de l'ensemble quotient : on regroupe les joueurs (
    # classes d'equivalence)
11 # selon la relation "a la meme couleur de maillot que"
12 equipes = {}
13
14 for j in joueurs:
15     c = j["couleur"]
16     if c not in equipes:
```

```

17     equipes[c] = [] # Creation d'une nouvelle classe d'
18     equivalence
19     equipes[c].append(j["id"])
20 print("Ensemble_Quotient_(Equipes_formees):")
21 for couleur, membres in equipes.items():
22     print(f"Classe_{couleur}:_Joueurs_{membres}")

```

2.4.7 Exercices à faire

1. **Exercice 1** : Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f(x) = x^2$. Cette fonction est-elle injective ? Surjective ? Bijective ? Justifiez rigoureusement.
2. **Exercice 2** : Sur l'ensemble des mots de la langue française, on définit la relation $\mathcal{R}$ par : " $mot_1 \mathcal{R} mot_2$ s'ils ont la même première lettre". Montrer que c'est une relation d'équivalence et décrire l'ensemble quotient.
3. **Exercice 3** : Démontrer que la composée de deux applications injectives est une application injective.

Pour aller plus loin : Ensembles, Relations et Intelligence Artificielle

Les notions de logique, d'ensembles et de relations ne sont pas de simples abstractions mathématiques : elles forment l'architecture logique sur laquelle reposent les bases de données modernes et les algorithmes d'Intelligence Artificielle. Pour clore ce chapitre, voici comment ces concepts prennent vie en Data Science.

Piste 1 : Théorie des Ensembles, Algèbre de Boole et Web Scraping

Lorsqu'un script d'extraction automatisée (web scraping) parcourt des plateformes d'emploi pour construire une base de données, il récolte un volume massif de texte brut. Transformer ce texte en données exploitables repose sur la théorie des ensembles.

Si l'on définit l'ensemble A comme les offres exigeant la compétence "Python", et B comme les offres proposant du "Télétravail", la requête permettant de trouver le job idéal est une simple **intersection** ($A \cap B$). À grande échelle, les moteurs de recherche (comme Elasticsearch) ou les bases de données relationnelles optimisent ces filtres en utilisant l'algèbre de Boole. Les lois de De Morgan, que nous avons démontrées, sont d'ailleurs utilisées par les optimiseurs de requêtes SQL pour réduire le temps de calcul lors du croisement de millions de tables.

Sujet de recherche : Explorez le concept d'**Indexation Inversée** (*Inverted Index*). Comment les moteurs de recherche utilisent-ils les opérations ensemblistes (unions et intersections de listes d'identifiants) pour renvoyer des résultats en quelques millisecondes ?

Piste 2 : Relations d'équivalence et Clustering (Apprentissage non supervisé)

En Machine Learning, on distingue l'apprentissage supervisé (où l'on connaît la réponse attendue) de l'apprentissage non supervisé. L'une des tâches phares du non super-

visé est le **Clustering** (ou partitionnement de données).

Prenons l'exemple de l'analyse vidéo d'un match de football par vision par ordinateur. L'algorithme détecte des dizaines de "points" en mouvement sur le terrain. Pour que l'analyse soit exploitable, l'IA doit grouper ces points. Mathématiquement, l'algorithme cherche à construire une **relation d'équivalence** $\mathcal{R}$ du type : " $x\mathcal{R}y$ si le joueur x et le joueur y portent une couleur de maillot similaire ou courent dans une formation similaire". En appliquant cette relation, l'algorithme génère un **ensemble quotient** : il sépare automatiquement l'ensemble des joueurs en classes d'équivalence distinctes (Équipe 1, Équipe 2, et les Arbitres).

Sujet de recherche : Renseignez-vous sur l'algorithme des **K-Means** (K-Moyennes) ou sur **DBSCAN**. Comment ces algorithmes créent-ils des partitions (classes d'équivalence) dans un espace de données à N dimensions en se basant sur une relation de "distance" mathématique ?

Piste 3 : Applications (Fonctions) et Modélisation

Tout modèle de Machine Learning est, fondamentalement, une **application** mathématique f qui relie un espace de départ E (les données d'entrée, ou *features*) à un espace d'arrivée F (les prédictions, ou *labels*).

- Si l'on souhaite que notre algorithme identifie de manière unique chaque utilisateur à partir d'une photo de son visage, nous exigeons que notre modèle f se rapproche le plus possible d'une application **injective** (deux visages différents ne doivent jamais donner le même identifiant).
- Si l'application ne peut prendre que des valeurs discrètes dans l'ensemble d'arrivée (ex : $F = \{\text{Spam}, \text{Non Spam}\}$), on parle d'un problème de **Classification**.

Sujet de recherche : Comment les réseaux de neurones artificiels agissent-ils comme des fonctions mathématiques complexes, et pourquoi étudie-t-on la notion d'espace vectoriel (notre prochain grand chapitre !) pour comprendre la dimensionnalité de E et F ?

Partie A : Entraînement technique et théorique

Exercice 1 : Logique formelle

1. Dresser la table de vérité de la proposition suivante : $P \implies (Q \vee R)$.
2. Soit f une fonction de $\mathbb{R}$ dans $\mathbb{R}$. Écrire la négation formelle de la proposition suivante :

$$\forall M \in \mathbb{R}, \exists A \in \mathbb{R}, \forall x \in \mathbb{R}, (x > A \implies f(x) > M)$$

3. Démontrer par l'absurde que si $A \cap B = A \cup B$, alors $A = B$.

Exercice 2 : Opérations sur les ensembles

Soient A , B et C trois sous-ensembles d'un ensemble de référence E .

1. Démontrer algébriquement la propriété d'absorption : $A \cup (A \cap B) = A$.
2. Montrer que : $(A \setminus B) \setminus C = A \setminus (B \cup C)$.
3. La différence symétrique est définie par $A \Delta B = (A \cup B) \setminus (A \cap B)$. Résoudre l'équation $A \Delta X = \emptyset$, d'inconnue $X \subset E$.

Exercice 3 : Dénombrement et Cardinaux

Soit E un ensemble fini de cardinal n .

1. Combien y a-t-il d'applications possibles de E vers un ensemble F de cardinal p ?
2. Combien y a-t-il de bijections possibles de E vers lui-même ?
3. Démontrer que le nombre total de sous-ensembles de E (le cardinal de $\mathcal{P}(E)$) est égal à 2^n . (*Indice : utiliser la formule du binôme de Newton vue au chapitre précédent*).

Exercice 4 : Applications et bijectivité

Soit l'application $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ définie par $f(x, y) = (x + y, x - y)$.

1. Montrer que f est injective.
2. Montrer que f est surjective.
3. En déduire l'expression de sa bijection réciproque $f^{-1}(x, y)$.

Partie B : Modélisation Algorithmique

Exercice 5 : Algèbre ensembliste et Web Scraping

Un script d'extraction automatisée est déployé pour récupérer des offres d'emploi sur le site *Tunisie Travail*. On note E l'ensemble total des offres extraites. On définit les sous-ensembles suivants :

- P : l'ensemble des offres mentionnant "Python".
 - D : l'ensemble des offres mentionnant "Data Analysis".
 - A : l'ensemble des offres mentionnant "Power Automate".
1. Exprimer à l'aide des opérations ensemblistes ($\cap, \cup, \setminus, \overline{}$) l'ensemble des offres qui requièrent Python et l'analyse de données, mais qui ne mentionnent pas Power Automate.
 2. Traduire en français la signification de l'ensemble suivant : $(P \cup D) \cap \overline{A}$.
 3. Si le script a extrait $|E| = 500$ offres, sachant que $|P| = 120$, $|A| = 40$ et que $P \cap A = \emptyset$, combien d'offres ne mentionnent ni Python ni Power Automate ?

Exercice 6 : Relations d'équivalence en Vision par ordinateur

Lors de l'analyse vidéo d'un match de football par un algorithme de traitement d'images, on note J l'ensemble des joueurs détectés sur le terrain à l'instant t . L'algorithme analyse les pixels de la zone délimitant chaque joueur et calcule une valeur $c(x)$ représentant le code couleur dominant de son maillot. On définit sur J la relation $\mathcal{R}$ par :

$$\text{Pour tous joueurs } x \text{ et } y, \quad x\mathcal{R}y \iff c(x) = c(y)$$

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur J .
2. Que représente concrètement la classe d'équivalence d'un joueur x (notée $\bar{x}$) sur le terrain ?
3. De combien d'éléments l'ensemble quotient $J/\mathcal{R}$ est-il composé lors d'un match classique (en supposant que les maillots des deux équipes et des arbitres sont parfaitement identifiés) ?

Travaux Pratiques (TP) Python

L'objectif de ce TP est d'implémenter les concepts de logique formelle, théorie des ensembles, dénombrement et relations sur machine. On exploitera les modules `itertools`, `math.comb`, `collections` et `matplotlib` (optionnel).

Exercice 1 : Tables de vérité automatisées

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire des fonctions `NEG(P)`, `ET(P, Q)`, `OU(P, Q)`, `IMP(P, Q)`, `EQUIV(P, Q)` qui prennent des booléens et retournent le résultat des connecteurs logiques.
2. **[Bloom : Application | Difficulté : Facile]**
Écrire `table_verite(formule, n)` qui prend une formule logique sous forme d'une fonction Python à n variables (ex. `lambda P, Q: IMP(P, Q)`) et imprime sa table de vérité complète sur les 2^n combinaisons.
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier automatiquement que `IMP(P, Q)` et `OU(NEG(P), Q)` ont la même table de vérité (équivalence logique). Vérifier aussi la contraposée : $P \implies Q$ et $\neg Q \implies \neg P$.

Exercice 2 : Détection de tautologies

1. **[Bloom : Synthèse | Difficulté : Moyenne]**
Une *tautologie* est une formule logique vraie pour toutes les valuations de ses variables. Écrire `est_tautologie(formule, n)` qui retourne `True` si la formule est une tautologie sur ses n variables.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier que les formules suivantes sont des tautologies :
 - $P \vee \neg P$ (tiers exclu) ;
 - $\neg(P \wedge Q) \iff (\neg P \vee \neg Q)$ (loi de De Morgan) ;
 - $((P \implies Q) \wedge (Q \implies R)) \implies (P \implies R)$ (transitivité de l'implication).

Exercice 3 : Opérations ensemblistes sans set

1. **[Bloom : Application | Difficulté : Facile]**
En représentant un ensemble par une liste sans doublons, écrire les fonctions `union(A, B)`, `intersection(A, B)`, `difference(A, B)`, `difference_symetrique(A, B)` sans utiliser le type `set` de Python.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier numériquement la *distributivité* de $\cap$ sur $\cup$: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ pour 100 triplets aléatoires d'ensembles d'entiers.
3. **[Bloom : Synthèse | Difficulté : Moyenne]**
Écrire `parties(E)` qui retourne l'ensemble des parties $\mathcal{P}(E)$ d'un ensemble fini. Vérifier que $|\mathcal{P}(E)| = 2^{|E|}$ pour $|E| \leq 8$.

Exercice 4 : Test des propriétés d'une relation

1. **[Bloom : Application | Difficulté : Moyenne]**
Une relation binaire sur un ensemble fini E peut être codée comme une fonction $r : E \times E \rightarrow \{0, 1\}$ ou comme un ensemble de couples. Écrire `est_reflexive(r, E)`, `est_symetrique(r, E)`, `est_transitive(r, E)`, `est_antisymetrique(r, E)`.

2. **[Bloom : Synthèse | Difficulté : Moyenne]**
En déduire `est_equivalence(r, E)` et `est_ordre(r, E)` (ordre = réflexive, antisymétrique, transitive).
3. **[Bloom : Création | Difficulté : Difficile]**
Tester sur $E = \{1, 2, 3, 4, 5, 6\}$:
— la relation « x divise y » : ordre ? équivalence ?
— la relation « $x \equiv y \pmod{3}$ » : ordre ? équivalence ?

Exercice 5 : Dénombrement et combinatoire

1. **[Bloom : Compréhension | Difficulté : Facile]**
Implémenter `factorielle(n)`, `arrangement(n, k)` ($A_n^k = n!/(n-k)!$) et `combinaison(n, k)` ($\binom{n}{k}$) sans utiliser `math`.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Comparer le résultat de votre `combinaison(20, 10)` à `math.comb(20, 10)`.
3. **[Bloom : Application | Difficulté : Moyenne]**
En utilisant `itertools.permutations`, `permutations(L, k)` (pour les arrangements) et `combinations(L, k)`, énumérer effectivement tous les arrangements et combinaisons de $\{a, b, c, d\}$ par $k = 2$ et compter pour vérifier les formules.

Exercice 6 : Principe d'inclusion-exclusion

1. **[Bloom : Application | Difficulté : Moyenne]**
Implémenter `cardinal_union(A, B)` qui calcule $|A \cup B|$ *uniquement* via la formule $|A| + |B| - |A \cap B|$ (sans construire l'union).
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Généraliser à trois ensembles : $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$. Écrire `cardinal_union_3(A, B, C)` et tester sur trois sous-ensembles aléatoires de $\{1, \dots, 100\}$.

Exercice 7 : Classes d'équivalence et partitions

1. **[Bloom : Application | Difficulté : Moyenne]**
Étant donnée une relation d'équivalence $\mathcal{R}$ sur E (codée comme un ensemble de couples ou une fonction), écrire `classes_equivalence(R, E)` qui retourne la liste des classes d'équivalence (= la *partition* associée).
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester sur $E = \{0, 1, 2, \dots, 11\}$ avec la relation $x\mathcal{R}y \iff x \equiv y \pmod{3}$.
Combien de classes ? Quels sont leurs cardinaux ?

Exercice 8 : Application — détection de doublons (web scraping)

1. **[Bloom : Création | Difficulté : Difficile]**
Vous récupérez une liste d'offres d'emploi sous forme de dictionnaires `{titre, entreprise, lieu, salaire}`. Vous voulez *regrouper les offres identiques* en utilisant la relation : « deux offres sont équivalentes si leurs (titre, entreprise) sont identiques après normalisation ».
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Écrire `normaliser(s)` qui passe une chaîne en minuscules et supprime les espaces superflus, puis `regrouper_offres(offres)` qui retourne la partition de

la liste sous forme de dictionnaire {(titre_norm, entreprise_norm) : [liste des offres associées]}.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

Tester avec une liste fictive de 10 offres dont 3 paires de doublons (par variation de casse ou d'espaces). Combien de classes après regroupement ?